

Acceso a Datos

Manejo de Conectores

Tema 3

1. Introducción

Base de datos relacional Aquella que presenta la información en **tablas con filas y columnas**.

Tabla / relación Colección de **objetos del mismo tipo (filas o tuplas)**. En cada tabla se elige una **clave primaria** para identificar de manera unívoca a cada fila de la misma.

Desfase objeto-relacional Diferencia de aspectos que existen entre la **programación orientada a objetos y la base de datos**.

2. Protocolos de acceso a bases de datos

La llegada del **ODBC** representó un avance en el camino hacia la interoperabilidad entre bases de datos y lenguajes de programación. La mayoría de empresas desarrolladoras de sistemas gestores de bases de datos **incorporaron los drivers de conectividad** a sus sistemas, y los principales lenguajes de programación desarrollaron **bibliotecas específicas** para soportar ODBC.

JDBC Java Database Connectivity. **API similar a ODBC**, implementada para el lenguaje Java, con funcionalidad **encapsulada en clases** y sin dependencia en ninguna plataforma específica. Permite **simplificar el acceso a bases de datos relacionales**.

Una aplicación JDBC está compuesta de **varias capas**, como se muestra en la [figura 1](#).

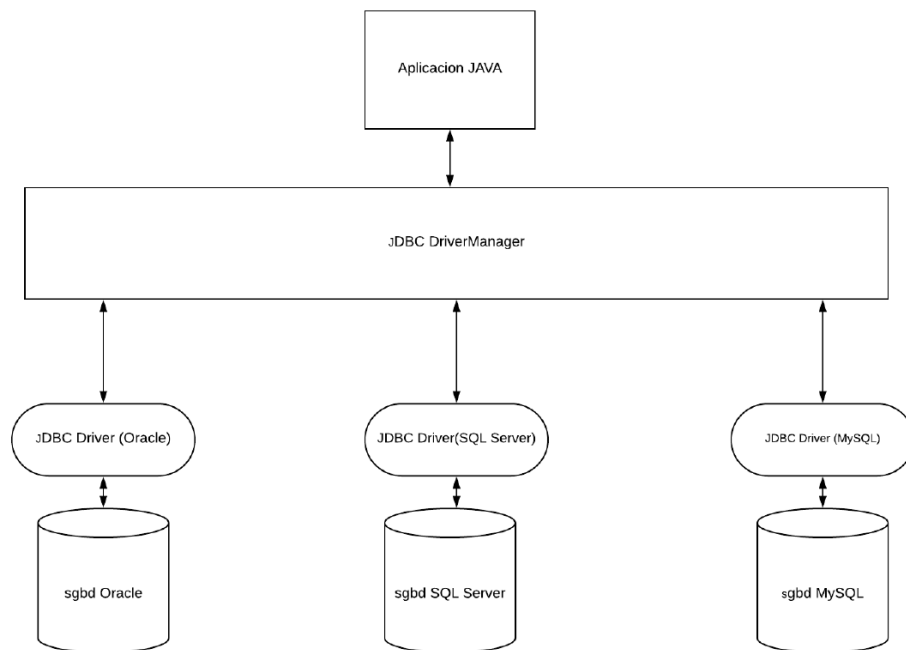


Imagen 1: Capas de una aplicación JDBC

Aplicación Java Capa superior. Puede comunicarse con muchas bases de datos con **pocas modificaciones**. JDBC proporciona una manera consistente de **conectarse a una base de datos, ejecutar comandos y recuperar los resultados**.

JDBC DriverManager Clase responsable de **localizar un controlador JDBC** que la aplicación necesita. Cuando una aplicación solicita una conexión, el **DriverManager** pregunta a cada controlador si puede conectarse a la base de datos solicitada.

Veamos un ejemplo de conexión a la base de datos:

```

1 import java.sql.Connection;
2 import java.sql.DriverManager;
3 import java.sql.ResultSet;
4 import java.sql.SQLException;
5 import java.sql.Statement;
6 public class ConsultaSencilla {
7     public static void main(String[] args) {
8         Connection conexion;
9         try {
10             String urljdbc = "jdbc:oracle:thin:@localhost:1521:XE";

```

```
11  conexion = DriverManager.getConnection(urljdbc, "ejemplo", ejemplo");
12  Statement smt = conexion.createStatement();
13  ResultSet rset = smt.executeQuery("select
14  empno, ename , job from emp order by 1");
15  while (rset.next())
16      System.out.println("empleado numero " + rset.getString(1) + " nombre " +
17      rset.getString(2) + " oficio " + rset.getString(3));
18      conexion.close();
19  } catch (SQLException e) {
20      e.printStackTrace();
21  }
22  }
23 }
```

3. Arquitectura JDBC

Modelo de dos capas Una aplicación se comunica **directamente con la fuente de datos**. Los comandos o instrucciones del usuario se envían a la base de datos y los resultados se devuelven al usuario. **Configuración cliente/servidor.**

Modelo de tres capas Los comandos se envían a una **capa intermedia de servicios**, la cual envía los comandos a la fuente de datos. Esta procesa los comandos y envía los resultados de vuelta a la capa intermedia, desde la que se envían al usuario.

3.1. Tipos de conectores

Tipo 1: JDBC-ODBC Bridge Proporcionan un puente entre el API JDBC y el API ODBC. Traduce las llamadas JDBD a ODBC y las envía a la fuente de datos ODBC. No necesita un driver específico por cada base de datos y está soportado por muchos fabricantes, pero hay plataformas que no lo tienen implementado y el rendimiento no es óptimo.

Tipo 2: API Nativa Convierten las llamadas JDBC a llamadas específicas de la base de datos. Se comunica directamente con el servidor de la base de datos, por lo que es necesario que haya código en la máquina cliente. Mejor rendimiento que el tipo 1, pero la librería de bases de datos necesita cargarse en cada máquina cliente.

Tipo 3: JDBC-Net pure Java Driver Aproximación de tres capas. Útil para aplicaciones en internet. Basado en servidor: no necesita librerías en las máquinas cliente. Soporte para balanceo de carga, auditoría, etc.

Tipo 4: Protocolo nativo Conectores que convierten directamente las llamadas JDBC al protocolo de red usado por el sistema gestor de la base de datos. Llamada directa desde la máquina cliente al servidor. No necesita traducción adicional, lo que mejora el rendimiento, y no se necesita instalar software especial. Sin embargo, cada base de datos necesita su propio driver.¹

4. Conexión a una base de datos

El método `DriverManager.getConnection` recibe como parámetro la URL de JDBC que identifica la base de datos a la que queremos conectar, y devuelve un objeto `Connection` que representa la **conexión** con la base de datos.

4.1. Drivers de conexión

4.1.1. MYSQL

Proporciona controladores basados en estándares para **JDBC, ODBC y .NET**. Para JDBC, proporciona conectividad para aplicaciones cliente con **MYSQL Connector / J**. Implementa JDBC, así como una serie de extensiones para el driver. Es un controlador **tipo 4**.

4.1.2. Oracle

JDBC Thin Driver usa **sockets de Java** para conectarse directamente a Oracle. **Independiente de la plataforma** y puede ejecutarse desde un **navegador web**.

JDBC OCI funciona a través de **SQL * Net**. Permite llamar al OCI directamente desde Java, lo que proporciona un **alto grado de compatibilidad** con una versión específica de Oracle.

JDBC KPRB se utiliza principalmente para **escribir procedimientos almacenados**. Utiliza la sesión de base de datos actual, por lo que no necesita un nombre de usuario, contraseña o URL adicional.

¹Aunque el tipo 5 aparece mencionado en el temario como “Controladores altamente funcionales con un rendimiento superior”, la especificación oficial solo menciona cuatro tipos. El tipo 5 solo es mencionado por la empresa Progress DataDirect en su página de ventas, no tiene ninguna referencia oficial, y no es conocida diferencia alguna con el tipo 4: <https://stackoverflow.com/questions/3989221/type-5-jdbc-driver>

4.2. Conexión con JDBC

La interfaz `Connection` representa **una conexión a la base de datos**. Los objetos mantendrán la capacidad de comunicarse con el sistema gestor mientras permanezcan abiertos.

El objeto `Connection` está totalmente vinculado a una fuente de datos, por lo que hay que **especificar la fuente** mediante la URL de los datos, y el usuario y contraseña de necesitarlos.

La forma más simple de URL JDBC es una **lista de tres valores** en formato `a : b : c` donde `a` es el **protocolo** (siempre `jdbc`), `b` es el **subprotocolo** y `c` es el **nombre de sistema para establecer la conexión** con un sistema específico.

4.3. Realización de consultas

Nuestra aplicación deberá:

- Cargar el driver necesario
- Establecer una conexión con la base de datos.
- Enviar consultas SQL y procesar el resultado.
- Liberar los recursos al terminar.
- Gestionar los errores que se puedan producir.

4.3.1. Consultas que devuelven datos

La consulta se realiza mediante `executeQuery`, los datos se devuelven mediante un `ResultSet`, y se accede a los datos mediante un cursor.

```
1 import java.sql.Connection;
2 import java.sql.DriverManager;
3 import java.sql.ResultSet;
4 import java.sql.SQLException;
5 import java.sql.Statement;
6 public class ConsultaSencilla {
7     public static void main(String[] args) {
8         Connection conexion;
9         try {
10             String urljdbc = "jdbc:oracle:thin:ejemplo/ejemplo@localhost:1521:XE";
11             conexion = DriverManager.getConnection(urljdbc, "ejemplo", "ejemplo");
12             Statement smt = conexion.createStatement();
13             ResultSet rset =
14                 smt.executeQuery("select
15                     empno, ename , job from emp order by 1");
16             while (rset.next()) {
17                 System.out.println("empleado numero " + rset.getString(1) +
```

```
18         " nombre " + rset.getString(2) + " oficio " + rset.getString(3));
19         resul.close();
20         smt.close();
21         conexion.close();
22     }
23     } catch (SQLException e) {
24         e.printStackTrace();
25     }
26 }
27 }
```

4.3.2. Consultas DML (UPDATE, DELETE, INSERT)

La consulta se realiza mediante `executeUpdate`, que devuelve un número entero que representa el **número de filas afectadas por la instrucción**.

```
1  import java.sql.Connection;
2  import java.sql.DriverManager;
3  import java.sql.SQLException;
4  import java.sql.Statement;
5  public class Actualizacion {
6      public static void main(String[] args) {
7          // establecemos la conexion
8          Connection conexion;
9          try {
10             String urljdbc = "jdbc:oracle:thin:ejemplo/ejemplo@localhost:1521:XE";
11             conexion = DriverManager.getConnection(urljdbc, "scott", "tiger");
12             // crea la sentencia
13             Statement stm = conexion.createStatement();
14             // ejecuta la actualizacion la ejecucion devuelve 1, numero de filas afectadas
15             System.out.println(stm.executeUpdate("insert into jobs values ('ID TEACH',
16                 'PROFESOR IT', 2500, 5000)"));
17             // valida los datos
18             // ejecuta el borrado, la ejecucion devuelve 19, numero de filas afectadas
19             System.out.println(stm.executeUpdate("delete from jobs "));
20             stm.close(); // Cerrar Statement
21             // cierra la conexion
22             conexion.close();
23         } catch (SQLException e) {
24             e.printStackTrace();
25         }
26     }
27 }
```

4.3.3. Sentencias preparadas

Prepared Statement Sentencia SQL de base de datos precompilada. Su ejecución será más **rápida que una SQL normal**, por lo que es adecuada cuando vamos a ejecutar **la misma sentencia muchas veces**.

Pondremos interrogantes donde irán los valores concretos que vayamos a insertar: cada interrogante **corresponde al número del orden en el que aparece**. Los métodos **set** permitirán rellenar los valores a posteriori.

```
1 import java.sql.Connection;
2 import java.sql.DriverManager;
3 import java.sql.PreparedStatement;
4 import java.sql.SQLException;
5 public class actualizaSentenciaPreparada {
6     public static void main(String[] args) throws SQLException {
7         String urljdbc = "jdbc:oracle:thin:@localhost:1521:XE";
8         Connection conexion = DriverManager.getConnection(urljdbc, "scott", "tiger");
9         PreparedStatement pstmt = conexion.prepareStatement("UPDATE EMP "
10             + "SET SAL = ? WHERE empno = ?");
11         pstmt.setInt(1, 1);
12         pstmt.setInt(2, 7844 );
13         pstmt.executeUpdate();
14         pstmt.close();
15         conexion.close();
16     }
17 }
```

4.3.4. Captura de errores y liberación de recursos

Las conexiones consumen muchos recursos, por lo que conviene **cerrarlas** mediante **Connection.close** cuando vayan a dejar de ser utilizadas. También conviene cerrar **los CallableStatement, las consultas y los resultados**.

La excepción más común de JDBC es **java.sql.SQLException**. Se puede producir tanto en **el controlador como en la base de datos**.

4.4. Transacciones

Tienen la característica de poder **deshacer los cambios efectuados** en las tablas, si no se han podido realizar todas las operaciones que forman parte de una transacción.

Garantizan **atomicidad, consistencia, aislamiento y durabilidad (ACID)**.

Una transacción tiene dos finales posibles: **COMMIT**, si se finaliza correctamente y sin problemas, o **ROLLBACK**, si hay un fallo y hay que deshacer los cambios efectuados hasta ese momento.

Para agrupar sentencias SQL en una única transacción:

- Desactivamos el autocommit con **conexion.setAutoCommit(false)**.
- Ejecutamos las sentencias.
- Finalizamos la transacción con **conexion.commit()**.

```

1 public void updateCoffeeSales(HashMap<String, Integer> salesForWeek)
2 throws SQLException {
3     PreparedStatement updateSales = null;
4     PreparedStatement updateTotal = null;
5     String updateString =
6         "update " + dbName + ".COFFEES " +
7         "set SALES = ? where COF_NAME = ?";
8     String updateStatement =
9         "update " + dbName + ".COFFEES " +
10        "set TOTAL = TOTAL + ? " +
11        "where COF_NAME = ?";
12    try {
13        con.setAutoCommit(false);
14        updateSales = con.prepareStatement(updateString);
15        updateTotal = con.prepareStatement(updateStatement);
16        for (Map.Entry<String, Integer> e : salesForWeek.entrySet()) {
17            updateSales.setInt(1, e.getValue().intValue());
18            updateSales.setString(2, e.getKey());
19            updateSales.executeUpdate();
20            updateTotal.setInt(1, e.getValue().intValue());
21            updateTotal.setString(2, e.getKey());
22            updateTotal.executeUpdate();
23            con.commit();
24        }
25    } catch (SQLException e) {
26        JDBCUtilities.printSQLException(e);
27        if (con != null) {
28            try {
29                System.err.print("Transaction is being rolled back");
30                con.rollback();
31            } catch (SQLException excep) {
32                JDBCUtilities.printSQLException(excep);
33            } finally {
34                if (updateSales != null) {
35                    updateSales.close();
36                }
37                if (updateTotal != null) {
38                    updateTotal.close();
39                }
40                con.setAutoCommit(true);
41            }
42        }
43    }

```

4.5. Consultas al diccionario de datos

La interfaz `DatabaseMetadata` permite obtener información sobre la base de datos en su conjunto.

getColumns Devuelve en un `ResultSet` la descripción de las columnas de la tabla.

getDriverName Devuelve en un `String` el nombre del controlador JDBC.

- getDriverVersion** Devuelve en un String el número de versión del controlador.
- getExportedKeys** Devuelve en un ResultSet la descripción de las columnas con clave foránea que hacen referencia a las columnas de clave principal de la tabla dada.
- getFunctions** Devuelve en un ResultSet la descripción del sistema y las funciones de usuario disponibles.
- getMaxConnections** Recupera el número máximo de conexiones simultáneas a esta base de datos.
- getPrimaryKeys** Devuelve en un ResultSet una descripción de las columnas de clave primaria.
- getProcedures** Recupera una descripción de los procedimientos almacenados disponibles.
- getSchemas** Devuelve en un ResultSet los nombres de esquema disponibles.
- getSQLKeywords** Recupera una lista separada por comas de todas las palabras clave de SQL de esta base de datos que no son también palabras clave de SQL 2003.
- getStringFunctions** Recupera una lista separada por comas de funciones de cadena disponibles.
- supportsAlterTableWithAddColumn** Devuelve un booleano determinando si esta base de datos admite añadir columnas con **ALTER TABLE**.
- supportsAlterTableWithDropColumn** Devuelve un booleano determinando si esta base de datos admite eliminar columnas con **ALTER TABLE**.
- supportsTransactions** Devuelve un booleano determinando si esta base de datos admite transacciones.

4.6. Pool de conexiones

Concepto estandarizado desde **JDBC 3.0**.

Consideremos un servidor web en el que hay 20 conexiones creadas y conectadas a la base de datos.

conexiones en espera = 20

Cuando llegan **5 peticiones web**, la aplicación pide **5 conexiones** al *pool*, quedando esperando en el *pool* **15 conexiones**.

$$\begin{array}{ccc} \text{conexiones en espera} & \xrightarrow{5} & \text{conexiones activas} \\ 15 & & 5 \end{array}$$

La operación **es muy rápida**, puesto que la conexión ya está creada y solo hay que marcarla como alguien la está usando.

Cuando las peticiones web finalizan, las conexiones no se cierran sino que se devuelven al *pool*.

$$\begin{array}{ccc} \text{conexiones en espera} & \xleftarrow{5} & \text{conexiones activas} \\ 20 & & 0 \end{array}$$

Si se piden más conexiones de las que hay esperando, se crearán nuevas conexiones hasta el máximo que permita el *pool*.

Cuando un objeto Java necesita una conexión, la solicita a través del método `dataSource.getConnection`. Será liberada mediante `connection.close`.

© 2026 - Esteban N. - estnicolas@proton.me

Este texto se encuentra licenciado bajo Creative Commons
Atribución-Reconocimiento/NoComercial-CompartirIgual 4.0 Internacional.

Para ver una copia de esta licencia visite:

<https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode.es>.



Ninguna parte de este documento, código o texto, ha sido realizada, apoyada, consultada, generada, corregida o involucrada en ninguna forma o contexto con inteligencia artificial, ni lo será en cualquier futura posible versión o corrección.

<https://notbyai.fyi>